



BlueHarbor Terminal — Documento tecnico

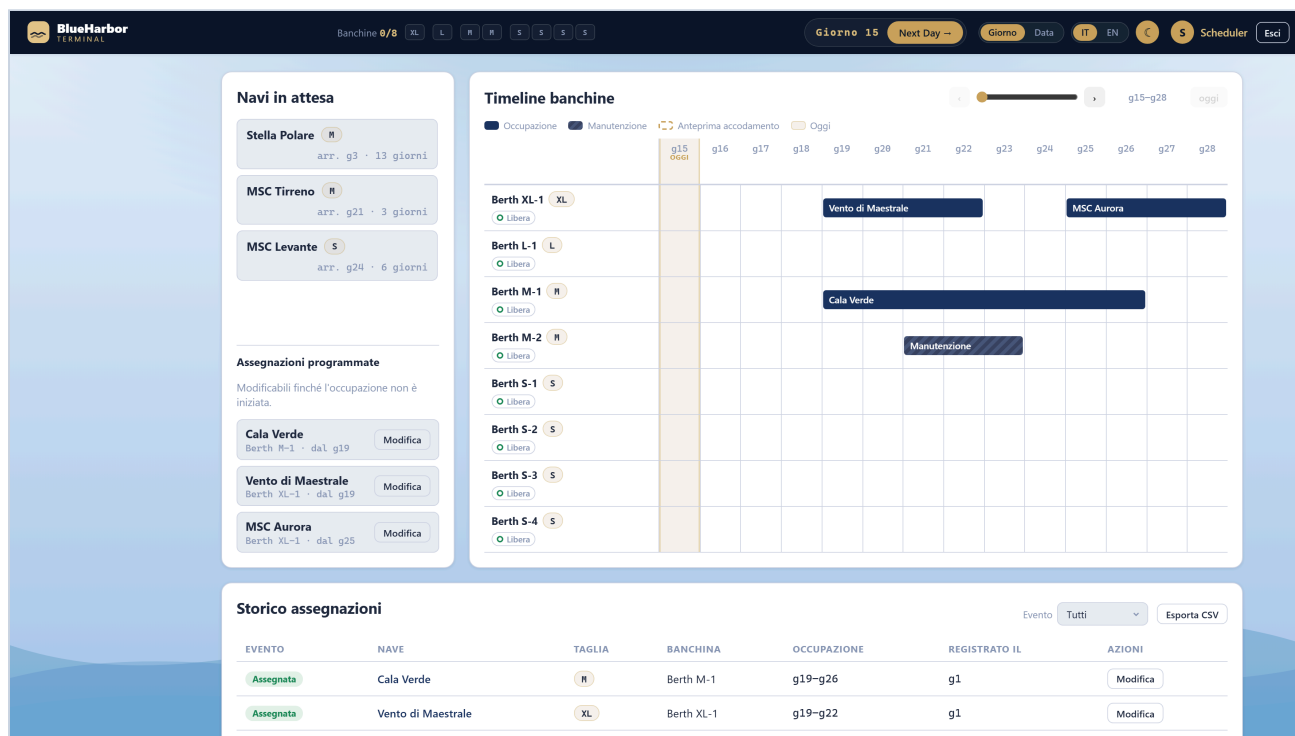
Prodotto: BlueHarbor Terminal · **Fornitore:** Q+2 (Quattrocchi + Due) · **Versione:** 1.1 — luglio 2026

Destinatari: referenti tecnici, IT del cliente, integratori. Per l'uso quotidiano: *Manuale d'uso*.

1. Sistema

Applicazione web per la gestione delle banchine di un terminal container. Copre registrazione delle navi, assegnazione a banchina con regole di compatibilità e accodamento, avanzamento del tempo su giorno virtuale, storico tracciabile.

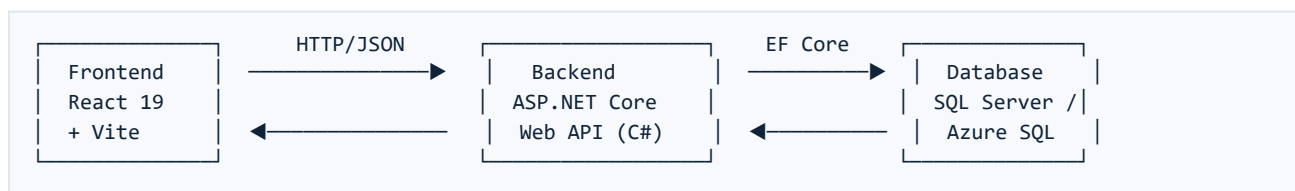
Multi-ruolo (Operatore, Scheduler, Admin). Distribuita come **istanza dedicata per cliente**, ospitata in cloud.



Vista Scheduler. A sinistra le navi in attesa e le assegnazioni programmate, a destra la timeline delle banchine.

1.1 Architettura

Tre livelli, processi separati, comunicazione esclusivamente **HTTP/JSON**.



Livello	Ruolo
Frontend	SPA React 19 servita da Vite. In sviluppo Vite fa da proxy su <code>/api</code> : origine unica per il browser.
Backend	Web API ASP.NET Core. Controller sottili, regole di dominio isolate in servizi. Contratto OpenAPI/Scalar.
Database	SQL Server / Azure SQL. Integrità applicata a livello di schema (<code>CHECK</code> , <code>FK</code>), non solo nel codice.

2. Stack

Livello	Tecnologia	Note
Frontend	React 19 + Vite	Stato globale via Context. Nessun router: la vista dipende dal ruolo.
Backend	ASP.NET Core Web API (C#, .NET)	Controller REST, servizi di dominio, middleware di autenticazione.
Accesso ai dati	Entity Framework Core	Modello EF allineato agli script SQL versionati.
Database	SQL Server / Azure SQL Database	Vincoli <code>CHECK</code> e chiavi esterne nello schema.
Autenticazione	JWT Bearer	Token firmato, scadenza 8 ore, ruoli applicati sugli endpoint.
Formato errori	Problem Details (RFC 7807)	Risposte d'errore uniformi.
Documentazione API	OpenAPI + Scalar	Esplorabile su <code>/scalar/v1</code> .
Packaging	Docker + docker-compose	DB e applicazione con un comando.

All'avvio l'applicazione garantisce schema e dati di base. L'operazione è idempotente su un database già inizializzato: l'avvio in container non richiede passaggi manuali.

3. Modello dati

Sei tabelle. `Ships` è il cuore del sistema, `AssignmentHistory` lo storico append-only.

Tabella	Colonne rilevanti	Vincoli
<code>Berths</code>	Id, Name, Size	<code>Size</code> $\in \{XL, L, M, S\}$ · Name univoco · 8 righe fisse (1 XL, 1 L, 2 M, 4 S).
<code>Ships</code>	Id, Name, Size, ArrivalDay, Duration, Status, BerthId (FK, null), OccupationStartDay (null), Notes	<code>Size</code> $\in \{XL, L, M, S\}$ · <code>Status</code> $\in \{Pending, Assigned, Departed\}$ · <code>ArrivalDay</code> ≥ 0 · $3 \leq Duration \leq 15$ · FK verso <code>Berths</code> .
<code>Settings</code>	Key (PK), Value	Chiave-valore. In uso: <code>CurrentVirtualDay</code> , <code>Day1Date</code> .
<code>BerthMaintenance</code>	Id, BerthId (FK), StartDay, EndDay, CreatedAt	Finestre di indisponibilità [<code>StartDay</code> , <code>EndDay</code>).
<code>Users</code>	Id, Username, PasswordHash, Role, IsActive	Username univoco · <code>Role</code> $\in \{Operator, Scheduler, Admin\}$ · <code>IsActive</code> falso $\Rightarrow$ niente login.
<code>AssignmentHistory</code>	Id, ShipId, ShipName, Size, BerthId, BerthName, OccupationStartDay, OccupationEndDay, EventType, EventDay, CreatedAt	Append-only · <code>EventType</code> $\in \{Assigned, Departed\}$ · snapshot denormalizzati.

3.1 Ciclo di vita di una nave

Stato	BerthId	OccupationStartDay	Transizione
Pending	NULL	NULL	Alla creazione da parte dell'Operatore.
Assigned	valorizzato	valorizzato	All'assegnazione da parte dello Scheduler.
Departed	resta valorizzato	resta valorizzato	All'avanzamento del giorno, a fine occupazione.

Una nave `Departed` conserva i dati di banchina a fini storici; lo slot risulta libero per le date successive.

3.2 Gestione dello schema

Script T-SQL incrementali: `script5.sql` (base), `script6` (storico), `script7` (ruolo Admin), `script8` (manutenzioni). Il modello EF Core va tenuto allineato agli script.

4. API REST

Tutti gli endpoint applicativi richiedono autenticazione; pubblici solo quelli di `auth`. La colonna Ruolo indica il ruolo minimo: l'Admin è sempre ammesso dove lo sono Operatore o Scheduler.

Metodo	Percorso	Ruolo	Descrizione
POST	/api/auth/login	pubblico	Verifica credenziali, rilascia il JWT.
GET	/api/auth/me	pubblico	Verifica della sessione all'avvio.
POST	/api/auth/logout	pubblico	Simbolico: con JWT stateless il client scarta il token.
GET	/api/system/current-day	autenticato	Giorno virtuale corrente e <code>day1Date</code> .
GET	/api/ships	Operatore	Elenco navi paginato e filtrabile, con conteggi per stato.
POST	/api/ships	Operatore	Crea una nave: nome obbligatorio, nota facoltativa; taglia, arrivo e durata generati.
PUT	/api/ships/{id}	Operatore	Corregge i dati di una nave non ancora ormeggiata.
DELETE	/api/ships/{id}	Operatore	Annula una nave solo se Pending , altrimenti 409 .
GET	/api/scheduler/dashboard	Scheduler	Navi Pending e stato di tutte le banchine in una sola risposta.
POST	/api/ships/{id}/assign	Scheduler	Valida la taglia, calcola il primo slot libero, registra lo storico.
POST	/api/ships/{id}/unassign	Scheduler	Annula un'assegnazione non ancora iniziata .
GET	/api/history	Scheduler	Storico assegnazioni in sola lettura, con filtri.
GET	/api/history/export	Scheduler	Stesso storico in CSV.
GET · POST	/api/admin/users	Admin	Elenco e creazione utenti (mai l'hash password).
PUT · DELETE	/api/admin/users/{id}	Admin	Ruolo, reset password, (dis)attivazione. Guardrail anti auto-lockout e ultimo Admin.
GET · POST	/api/admin/maintenance	Admin	Finestre <code>[inizio, fine)</code> . 409 se incrociano navi assegnate o altre finestre.
DELETE	/api/admin/maintenance/{id}	Admin	Revoca una finestra non ancora iniziata, altrimenti 409 .
POST	/api/admin/simulation/reset	Admin	Riporta a giorno 1: cancella navi e storico, lascia banchine e utenti.
POST	/api/time/next-day	autenticato	Avanza il giorno, rilascia le navi concluse, registra le partenze.

5. Logica di dominio

Le regole sono isolate in servizi dedicati, testabili in modo indipendente dall'infrastruttura.

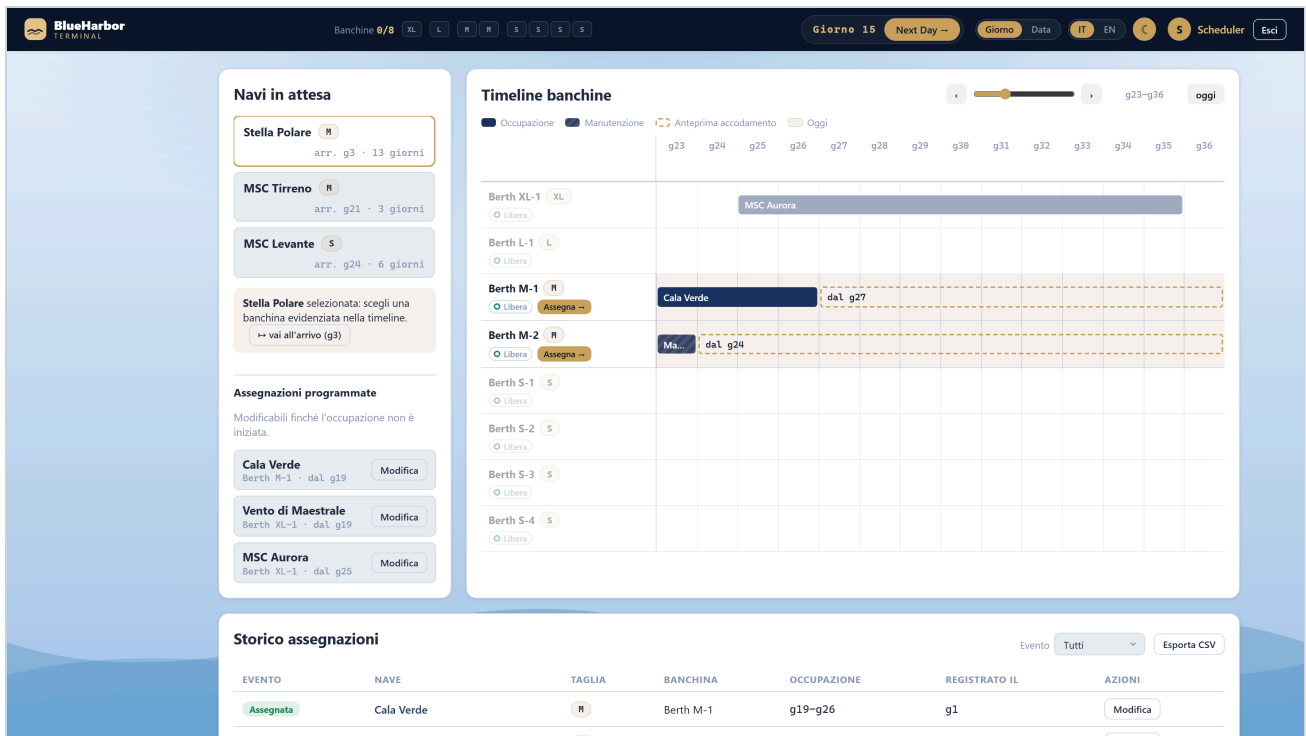
5.1 Compatibilità e accodamento

Due regole governano l'assegnazione.

1. **Compatibilità di taglia** — una nave entra solo in una banchina della propria taglia.
2. **Primo slot libero** — l'occupazione parte dal primo giorno disponibile della banchina, mai prima dell'arrivo della nave e mai nel passato.

Esempio. Nave A arriva il giorno 2, dura 5 giorni: occupa 2–6. Nave B arriva il giorno 3, dura 4 giorni: l'arrivo cade dentro l'occupazione di A, quindi B viene accodata e parte dal giorno 7.

Il calcolo è realizzato da funzioni pure (`SchedulingRules`): nessun accesso al database, nessuna dipendenza dall'interfaccia. È la ragione per cui è coperto da test automatici significativi.



Selezionata una nave, la timeline evidenzia le sole banchine compatibili e mostra tratteggiata l'anteprima del primo giorno libero. Il blocco pieno è un'occupazione, quello rigato una manutenzione.

5.2 Tempo virtuale

Il tempo è un intero (`Settings.CurrentVirtualDay`), non un orologio reale. L'operazione **Next Day**, in **un'unica transazione**:

- incrementa il contatore del giorno;
- marca `Departed` le navi `Assigned` la cui occupazione è terminata ($\text{OccupationStartDay} + \text{Duration} \leq \text{nuovo giorno}$), liberando le banchine.

L'avanzamento del giorno **non assegna mai** navi: l'assegnazione è sempre esplicita.

5.3 Manutenzioni

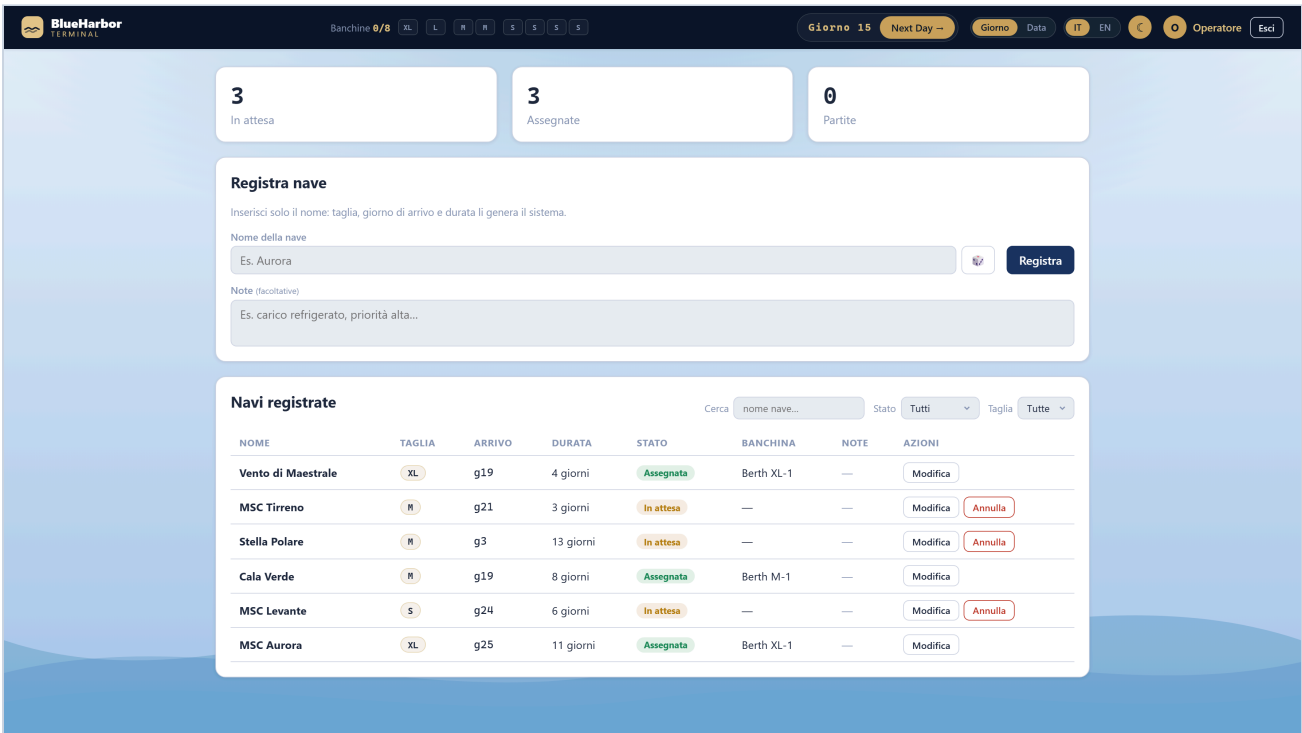
Una finestra rende la banchina indisponibile in `[StartDay, EndDay)`, con la stessa semantica di un'occupazione: l'algoritmo di accodamento non distingue i due casi e non è stato modificato per accoglierle. Il sistema **rifiuta** una manutenzione sovrapposta a una nave già assegnata o a un'altra finestra — le assegnazioni confermate non vengono mai alterate.

6. Sicurezza

Aspetto	Implementazione
Autenticazione	JWT firmato, scadenza 8 ore, inviato dal client a ogni richiesta.
Autorizzazione	Ogni controller applicativo protetto: <code>401</code> senza token valido, <code>403</code> con ruolo insufficiente. Pubblici solo gli endpoint <code>auth</code> .
Ruolo Admin	Ruolo di piattaforma: somma Operatore e Scheduler, aggiunge utenti e manutenzioni. Nessun potere di dominio aggiuntivo — niente riassegnazioni, nessun bypass delle regole, nessuna modifica al set di banchine.
Password	PBKDF2, salt casuale, 100.000 iterazioni. Mai in chiaro.
Errori	Handler globale: le eccezioni non gestite diventano un <code>500</code> in Problem Details, senza stack trace verso il client.
Segreti	Connection string e chiave JWT come <i>secret</i> dell'infrastruttura, passati via variabili d'ambiente. Mai nel repository né nell'immagine.

7. Frontend

React 19 + Vite. **Nessun router:** App sceglie fra `OperatorView`, `SchedulerView` e `AdminView` in base al ruolo autenticato. Ogni vista è autosufficiente; in `components/` restano solo i componenti realmente condivisi (barra superiore, notifiche, spinner).



Vista Operatore. Contatori per stato, modulo di registrazione, tabella filtrabile e paginata.

7.1 Stato globale

Contesto	Responsabilità
<code>AuthContext</code>	Utente e ruolo, login/logout, token in <code>localStorage</code> .
<code>DayContext</code>	Giorno virtuale letto dal backend; le viste si ricaricano quando cambia.
<code>ToastContext</code>	Notifiche di esito.

7.2 Servizi condivisi

Modulo	Responsabilità
api.js	Unico punto di chiamata HTTP: token, mappatura Problem Details → messaggio, download CSV.
scheduling.js	Replica client dell'algoritmo di accodamento, solo per l'anteprima visiva.
roles.js	Ruoli ed etichette, unico punto di verità.

L'anteprima di accodamento nel frontend è indicativa. Fa fede il giorno restituito dal server al momento dell'assegnazione.

Programma manutenzione

La banchina non sarà utilizzabile nei giorni indicati. Le navi assegnate dopo si accodano oltre la finestra. Le navi **già assegnate non si spostano**: se la finestra le incrocia, la richiesta viene rifiutata.

Banchina: Data inizio: Data fine (inclusa):

Giorno corrente: g15

Manutenzioni

BANCHINA	FINESTRA	STATO	AZIONI
Berth XL-1	g46 - g54	programmata	Revoca
Berth M-2	g21 - g23	programmata	Revoca
Berth S-2	g4 - g7	conclusa	Revoca

Vista Admin. Programmazione di una finestra di manutenzione ed elenco delle finestre con il relativo stato.

8. Infrastruttura e rilascio

Istanza dedicata su Microsoft Azure.

Componente	Servizio
Hosting applicativo	Azure Container Apps, una replica sempre attiva, HTTPS terminato dalla piattaforma.
Database	Azure SQL Database.
Telemetria	Application Insights, dove è configurata la connection string.

8.1 Pipeline

A ogni aggiornamento della linea principale, GitHub Actions:

1. esegue la suite di test come *gate* — test rossi, rilascio fermo;
2. costruisce l'immagine Docker;
3. la pubblica su GHCR, taggata per commit;
4. aggiorna l'istanza in cloud, che dichiara la versione servita.

Lo stesso risultato è riproducibile in locale con `docker compose up --build` (DB e applicazione su una macchina sola): utile per collaudo e dimostrazioni offline.

8.2 Deployment per cliente

Ogni cliente riceve un'istanza isolata, con database e utenti propri. Numero e taglie delle banchine e utenti iniziali si impostano in fase di attivazione.

9. Qualità e verifiche

- **Test automatici** — suite xUnit sul cuore logico (accodamento, hashing delle password), eseguita nella verifica standard e come *gate* del rilascio.
- **Vincoli nello schema** — taglie, stati e durate espressi anche come `CHECK` /FK nel database, non solo nel codice applicativo.
- **Contratto API documentato** — ogni controller documenta il proprio contratto; l'API è esplorabile via OpenAPI/Scalar.

10. Limiti noti e direzione

Limiti attuali

Limite	Stato
Multi-tenancy	Non nativa: oggi un porto = un'istanza dedicata.
Set di banchine	Fisso (8 banchine, taglie predefinite), impostato in configurazione.
Registrazione utenti	Riservata all'Admin, nessuna auto-registrazione.
Copertura test	Concentrata sulla logica di dominio. I test d'integrazione sui controller sono un'estensione pianificata.
Rete e identità in cloud	Accesso al database via firewall anziché endpoint privato; pipeline autenticata con service principal anziché identità federata; nessun controllo automatico dopo la pubblicazione.

Aggiornamenti previsti

Aggiornamento	Contenuto
UX e accessibilità	Interfaccia più rapida in uso operativo; completamento della conformità WCAG AA, navigazione da tastiera e lettori di schermo.
Multi-porto	Gestione di più terminal da un'unica console, con separazione dei dati per porto.
Report avanzati	Indicatori di occupazione delle banchine e andamento nel tempo, oltre all'export CSV già disponibile.